Module 10 — Data Visualization with Matplotlib

Why Data Visualization is Important

In data analysis, raw numbers are difficult to interpret.

Example:

Revenue = [100, 200, 300, 250, 400]

It is difficult to immediately identify:

• Trends
• Patterns
• Growth or decline
• Comparisons

Solution: Data Visualization

Visualization converts numerical data into graphical form.

This helps in:

• Understanding trends quickly
• Communicating insights effectively
• Supporting decision-making

Real-World Applications

• Business dashboards
• Sales trend analysis
• Financial reporting
• Machine learning model evaluation

Why Matplotlib?

Matplotlib is a **core Python library for data visualization**.

Key Advantages

• Works seamlessly with NumPy and Pandas
• Highly customizable

- Supports a wide variety of plots
- Foundation for other libraries like Seaborn

When to Use Matplotlib vs Seaborn

Matplotlib:

- Full control over plots
- Low-level customization
- Suitable for building plots from scratch

Seaborn:

- High-level interface
- Better aesthetics by default
- Used for statistical visualizations

Conclusion

Matplotlib is the **foundation of visualization in Python**.

All advanced visualization libraries are built on top of it.

Anatomy of a Matplotlib Figure

Understanding structure is critical.

---

1. Figure

The entire canvas or window where the plot is drawn.

---

2. Axes

The actual plot area where data is visualized.

A figure can have multiple axes.

---

3. Axis

Refers to:

- X-axis (horizontal)
- Y-axis (vertical)

Structure Summary

Figure → contains Axes

Axes → contains Axis

Visualization Flow

1. Create Figure
2. Create Axes
3. Plot data
4. Customize plot

Basic Plotting Workflow

The standard workflow in Matplotlib is:

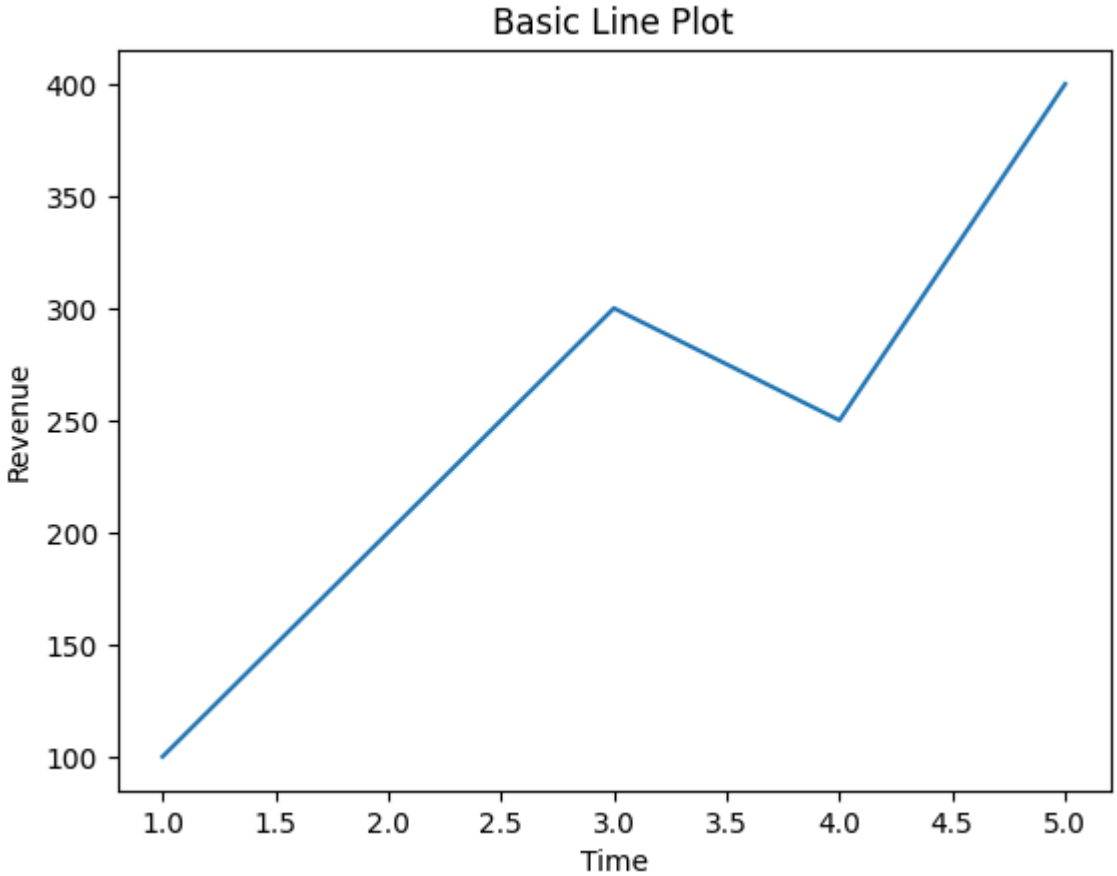
1. Import library
2. Prepare data
3. Create plot
4. Add labels and title
5. Display plot

Syntax Overview

import matplotlib.pyplot as plt

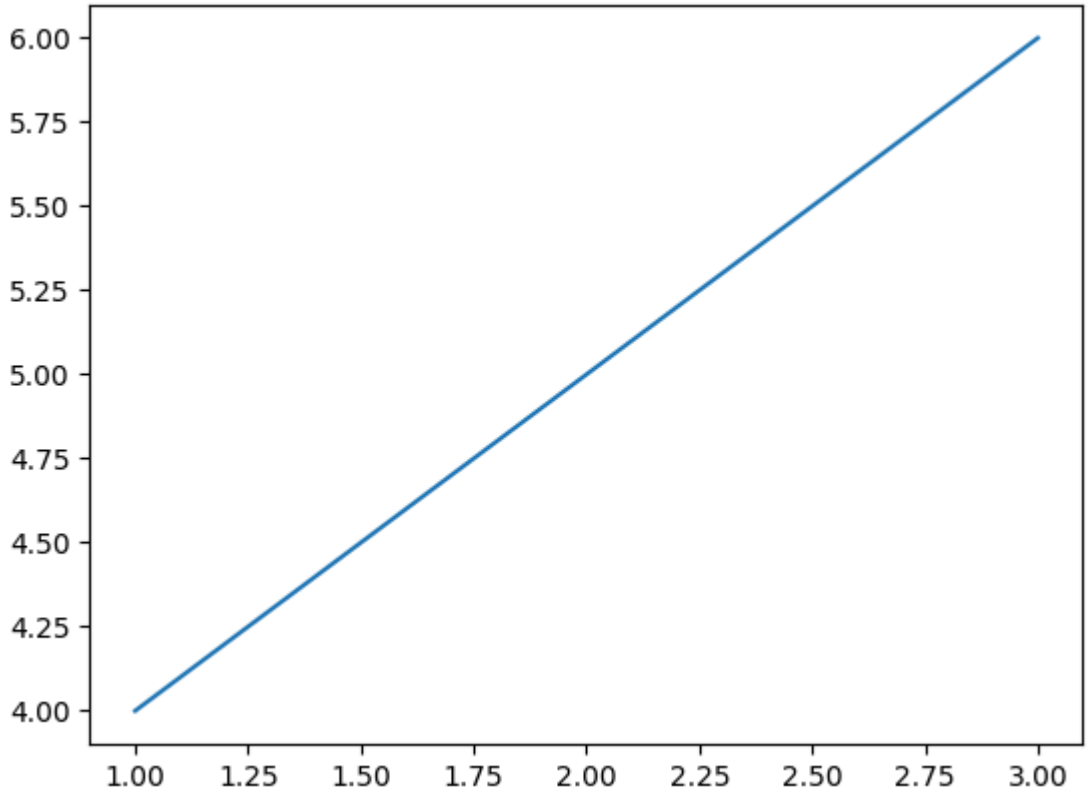
- plt.plot(x, y)
- plt.title("Title")
- plt.xlabel("X Label")
- plt.ylabel("Y Label")
- plt.show()

```
import matplotlib.pyplot as plt

# Sample data
x = [1,2,3,4,5]
y = [100,200,300,250,400]

# Create plot
plt.plot(x, y)

# Labels and title
plt.title("Basic Line Plot")
plt.xlabel("Time")
plt.ylabel("Revenue")

# Display
plt.show()
```



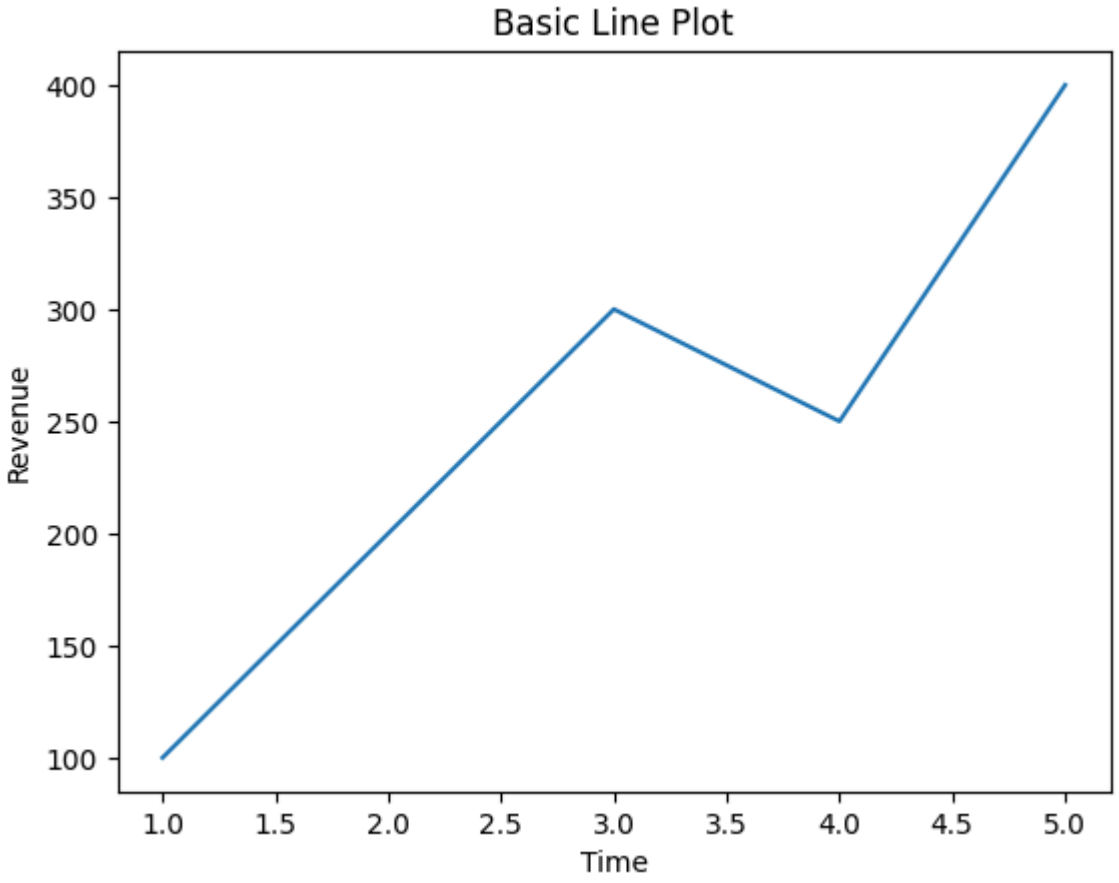

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 3], [4, 5, 6])  # creates the plot
# plt.show()                      # displays it
```

```
[<matplotlib.lines.Line2D at 0x7cc14dba55e0>]
```




```
x = [1,2,3,4,5]
y = [100,200,300,250,400]

# Create plot
plt.plot(x, y)

# Labels and title
plt.title("Basic Line Plot")
plt.xlabel("Time")
plt.ylabel("Revenue")
```

```
Text(0, 0.5, 'Revenue')
```



SESSION 2 — Line Charts (plt.plot)

Purpose

Line plots are used to show:

- Trends over time
- Continuous data
- Growth or decline

Syntax

plt.plot(x, y)

Parameters

x → values for x-axis
y → values for y-axis

Default Behavior

- Line color → blue
- Line style → solid
- No markers

Important Parameters of plt.plot()

plt.plot(x, y, color=None, linestyle='-', linewidth=1, marker=None, label=None)

---

1. color

Defines color of line

Examples:

- 'red'
- 'blue'
- 'green'
- 'black'

---

2. linestyle

Defines style of line

Default = '-'

Options:

- '-' → solid
- '--' → dashed
- ':' → dotted
- '-.' → dash-dot

---

3. linewidth

Thickness of line

Default = 1

Example:

linewidth=2

---

4. marker

Shows data points

Examples:

- 'o' → circle
- 's' → square
- '^' → triangle

---

5. label

Used for legend

1. Basic Shape Markers

These are the most commonly used markers in plots.

Marker	Description
'o'	Circle
's'	Square
'^'	Triangle pointing up
'v'	Triangle pointing down
'<'	Triangle pointing left
'>'	Triangle pointing right
'd'	Diamond
'D'	Thin diamond

---

2. Point and Pixel Markers

These markers are useful when working with large datasets.

Marker	Description
'.'	Point
','	Pixel

---

3. Plus, Cross and Line Markers

These markers are commonly used in technical or minimal plots.

Marker	Description
'+'	Plus
'x'	Cross
'X'	Filled cross
'	'
'_'	Horizontal line

---

4. Star and Special Markers

These markers help in visually distinguishing important data points.

Marker	Description
'*'	Star
'p'	Pentagon
'h'	Hexagon
'H'	Alternate hexagon
'8'	Octagon

---

5. Tick and Caret Markers

These are less commonly used but can be useful in specific visualizations.

Marker	Description
'1'	Tri-down tick
'2'	Tri-up tick
'3'	Tri-left tick
'4'	Tri-right tick
'P'	Filled plus

---

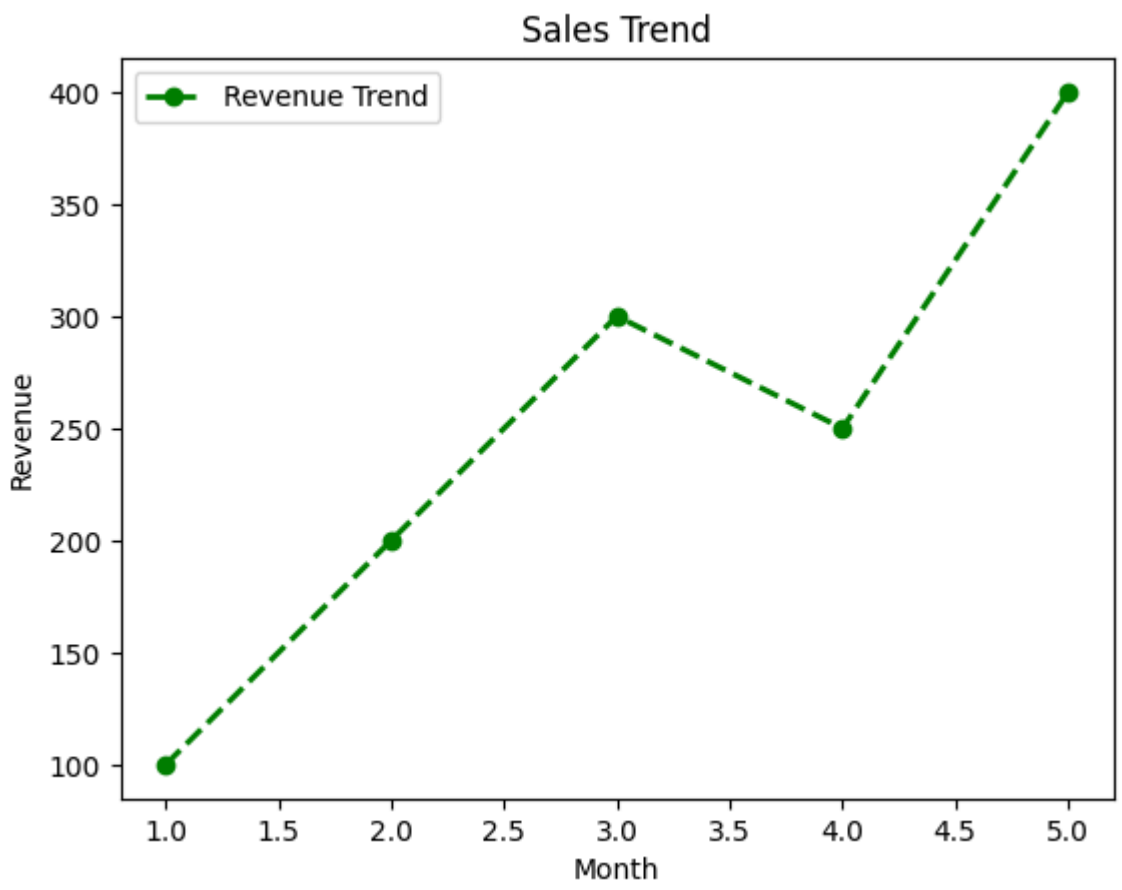
6. Special Cases

Marker	Description
'None' or ' '	No marker
'$text$'	Custom text marker

Example of custom marker:

```
plt.plot(x, y, marker='$A$')
```

```
x = [1,2,3,4,5]
y = [100,200,300,250,400]

plt.plot(
    x,
    y,
    color='green',
    linestyle='--',
    linewidth=2,
    marker='o',
    label="Revenue Trend"
)

plt.title("Sales Trend")
plt.xlabel("Month")
plt.ylabel("Revenue")

plt.legend()

plt.show()
```



Demo — Sales Trend Chart

Scenario

We want to visualize sales over time.

Why Line Chart?

Because:

- Data is continuous
- We want to see trend

```
months = ["Jan","Feb","Mar","Apr","May"]
sales = [100,200,150,300,250]

plt.plot(
    months,
    sales,
    marker='o',
    linestyle='-',
    color='blue',
    linewidth=2
)

plt.title("Monthly Sales Trend")
plt.xlabel("Months")
plt.ylabel("Sales")

plt.show()
```

Question 1

Create a simple line plot using:

x = [1,2,3,4,5]
y = [10,20,30,40,50]

Tasks:

- Plot the data
- Display the graph

Question 2

Using the same data:

- Add title "Simple Line Plot"
- Label x-axis as "X Values"
- Label y-axis as "Y Values"

Question 3

Create a line plot:

y = [5, 15, 25, 35, 45]

Tasks:

- Do not pass x values
- Observe what Matplotlib uses as default x

Question 4

Plot the following:

x = [1,2,3,4]
y = [100,200,150,300]

Tasks:

- Change line color to red
- Increase linewidth to 3

Question 5

Using same data:

- Change linestyle to dashed
- Add marker 'o'

Question 6

Plot:

x = [1,2,3,4,5]
y = [10,30,20,40,25]

Tasks:

- Use green color
- Use dotted line
- Add square markers

Question 7

Create a sales dataset:

months = ["Jan","Feb","Mar","Apr"]
sales = [100,200,150,300]

Tasks:

- Plot the data
- Add proper title and axis labels